
CSC 406 – Computer Graphics

Programming Assignment 03, Fall 2023

Monday, October 23rd, 2023

Due date: Thursday, November 3rd

1 What this Assignment is About

1.1 Objectives

In this assignment, we have three objectives:

- To implement simple picking and bounding box algorithms;
- To complete simple 2D games.

This is an individual assignment. You are of course encouraged to discuss it with other students but I expect you complete it by yourself.

I think that, the way the assignment is set up, it should be possible to get an A grade by doing only one of the projects with all the EC. I will recalculate to make sure and will add more EC if needed.

1.2 Handouts

The only handout for this assignment is this document, as a pdf file.

1.3 Hierarchical Collision Boxes

1.3.1 Hierarchical boxes

The almost universal strategy to perform efficiently the numerous tests for collision required of most games or simulation applications is to use a hierarchy of collision boxes (or bounding boxes). Here I use the term loosely, as these boxes could be rectangles or circles in 2D, or rectangular boxes, cylinders, or spheres in 3D. Simple box shapes are used because the tests can be performed very fast, as we saw in class and in the last assignment. Using a *hierarchy* of bounding boxes may seem questionable at first, even the object we are looking at is made up of many parts, like the one shown in Figure 1

The justification comes from the algorithmic complexity of the collision detection problem in a scene. If your scene comprises N objects that may collide with each other, then you need to perform $N \cdot (N - 1) = O(N^2)$ collision detection tests between the objects. If these objects are all made up of n different parts, then you now have to test for collision between $n \cdot N$ objects, a $N \cdot (N - 1) = O(n^2 N^2)$ problem.

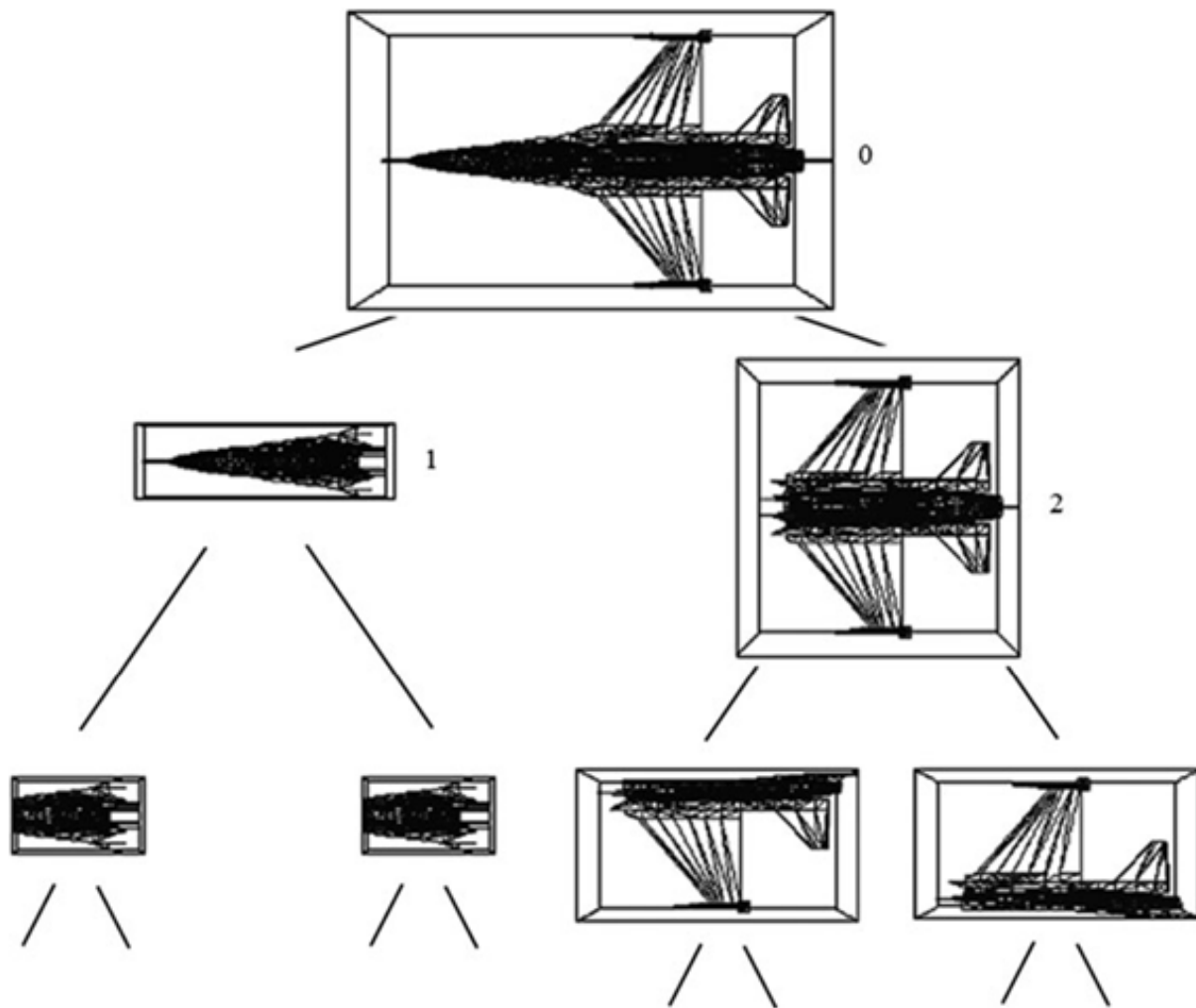


Figure 1: A bounding box hierarchy for an object made up of many parts

Assume that your object is made up of 8 parts, organized into two groups. So, we have 8 boxes, one for each part, then two group boxes, and finally, a box for the entire object. When testing for collision, instead of having to test for the 8 individual parts' box each and every time, we test with the global box first, and only test with the two group boxes if there is collision with the global box. In turn, we only test with a part's box if its group box has been hit. So, in the worst case we may have to perform $1 + 1 + 4$ tests (which is still better than 8), but most of the time we will only have to perform the test on the global box (because objects don't collide with each other *all the time*).

In the case of our silly face objects, the gain from hierarchical boxes won't be spectacular¹ because we only have three parts, fairly close to each other as well. We end up with boxes for the left ear, right ear, and the face proper, and one global box for the entire object, as shown in Figure 2

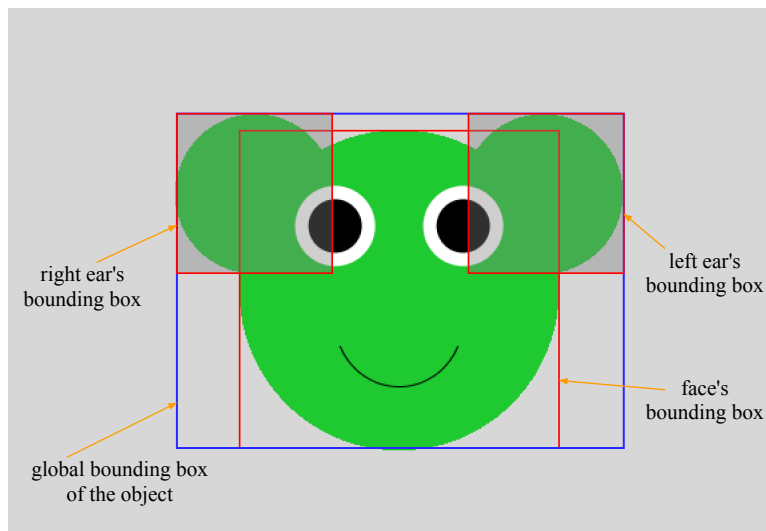


Figure 2: A bounding box hierarchy for the `Face` objects.

1.4 Absolute vs. relative boxes

“Absolute” bounding boxes are defined in the world reference frame, with their directions parallel with that of the window (Figure 3: Left). By opposition, “relative” bounding boxes are defined in the object's own reference frame and therefore rotate with the object (Figure 3: Right).

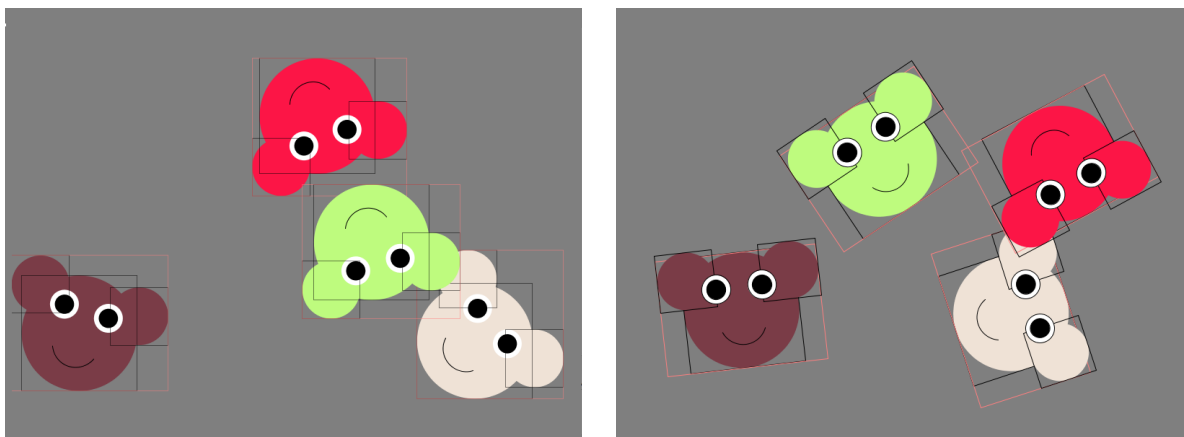


Figure 3: Bounding boxes for the `Face` objects. Left: absolute boxes. Right: relative boxes.

¹To begin with, because all the parts are circular, the use of collision circles would have been much more appropriate, and faster as well. However, in the general case, the automatic computation of circular bounding boxes is very tricky.

Both kinds of bounding have their respective advantages and disadvantages. Sometimes, both kinds are used in the same application (see Figure 4).

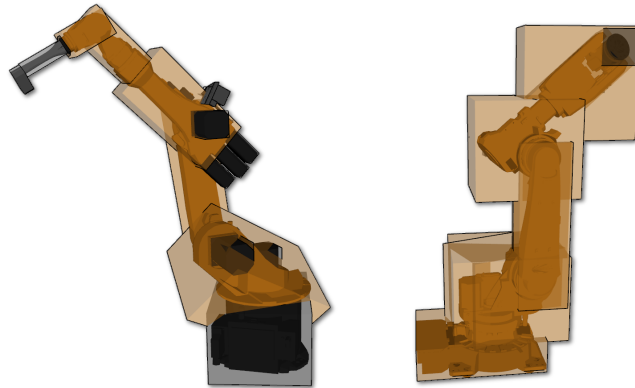


Figure 4: Absolute and relative bounding boxes in a robotics application.

2 Part I: Frogger

2.1 The game

You have probably heard of (and maybe even played) Frogger, one of the classical arcade and console games. If not, then you can watch this video: https://www.youtube.com/watch?v=WNrz9_Fe-Us

2.2 What you need to implement

2.2.1 The road

Your road will be a bi-directional 2-lane-per direction affair. In both direction there will be a slower lane on the lower side and a faster lane on the upper side. Cars on a given lane will all be moving at the same speed, but the spacing between car will be random.

2.2.2 Graphic objects and bounding boxes

Let's keep it simple. Maybe make the cars and Frogger somewhat distinct from plain rectangles, but don't get overboard: This is not the purpose of this assignment. What I care about are the bounding boxes, which are going to be rectangles for the cars and Frogger.

2.2.3 Game and score

The frogger character will have a fixed number of lives (let's say three, to keep the grading in finite time). After the frogger has been squashed for the last time, the game is over.

The score is based on the time taken to cross the road. While the game is playing, you should display the current score.

2.2.4 Controls

Use the arrow keys to control the displacements of Frogger. Please note that you cannot use the regular `glutKeyboardFunc()` callback setup to handle the arrow keys. Instead you must use the `glutSpecialFunc()` callback setup (and `GLUT_KEY_UP`, `GLUT_KEY_LEFT`, etc. for the key code, an `int` rather than an `unsigned char`).

2.3 Playability

I won't tell you how fast your cars and Frogger should move. This is something that you have to tune so that the game is playable and not too easy.

2.4 Extra credit

2.4.1 EC1: circular bounding box (6 pts)

If you give a circular bounding box to Frogger, then you need to perform circle vs. rectangle intersection tests to handle collision detection. This is only marginally more complex on your end, but this should make it a little easier for Frogger to escape getting flattened.

2.4.2 EC2: Add some animation to Frogger (2 to 8 points)

Frogger shouldn't just be anymore a rigid block of pixels moving around. There should be some form of change happening to the critter's appearance.

2.4.3 EC3: Flattened Frogger (2 to 6 points)

When Frogger gets killed, you can display for a couple of seconds a different rendering of your Frogger object. This will give you the lower score for this EC.

If you want the full mark, then you should produce an *animation* of Frogger getting run over.

3 Part II: Asteroids

This is another classical game. The animation of spaceship is a bit different from the ones I can see videos for online. I will post a video of the expected behavior in a couple of days.

3.1 What the spaceship looks like

It shouldn't be too hard for you to come up with a better spaceship model than the one shown on the left side of Figure 5. The right side of the figure shows how my model was built: The body of the spaceship is made up of two triangles and the thrusters are two rectangles. This model would have only two levels in its box hierarchy. It shouldn't be too hard for you to improve on this and provide a model with a three-levels box hierarchy. There will be up to 5 points of extra credit for particularly nice or cool-looking spaceships.

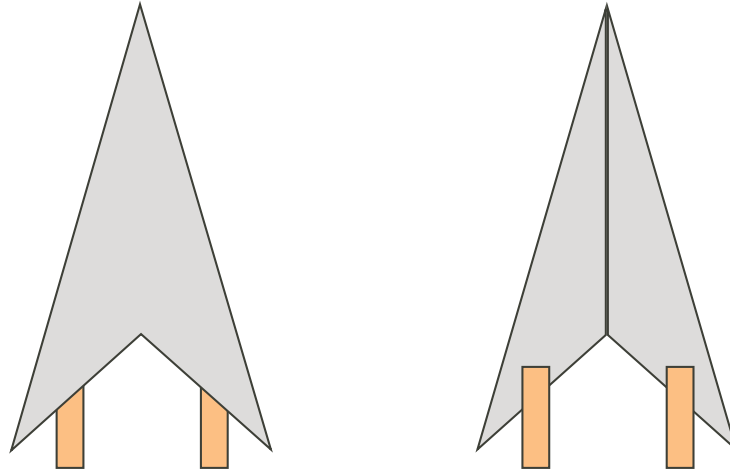


Figure 5: Left: a simple spaceship model. Right: How it is built.

3.2 Animation of the spaceship

This is an example of physics-based animation. The *state vector* of the spaceship stores the position and orientation of the spaceship, as well as the components of its velocity vector:

$$\mathbf{s}(t) = \begin{pmatrix} x(t), & \text{horizontal coordinate in world units,} \\ y(t), & \text{vertical coordinate in world units,} \\ \theta(t), & \text{angle in radians relative to the horizontal direction}^2, \\ v_x(t), & \text{horizontal component of the speed vector,} \\ v_y(t), & \text{vertical component of the speed vector,} \end{pmatrix} \quad (1)$$

Note that spaceship may not be pointed in the direction in which it is gliding/drifting. Expressed mathematically, this means that *a priori*

$$\theta(t) \neq \text{atan2}(v_y(t), v_x(t)).$$

The user controls the spaceship by

- Adjusting the direction in which it is pointing, by applying an *angular velocity* $\omega(t)$ to the spaceship, to make it spin around its center;
- Applying thrust, that is an acceleration vector $\vec{a}(t)$ in the direction in which it is pointing.

The effect of these controls is summarized in Figure 6. Because the acceleration vector $\vec{a}(t)$ has the same direction $\theta(t)$ as the the spaceship, this means that we really only control the *magnitude* of $\vec{a}(t)$, $a(t)$:

$$\vec{a}(t) = a(t) \begin{pmatrix} \cos \theta(t) \\ \sin \theta(t) \end{pmatrix}. \quad (2)$$

This means that the *state equation* for our spaceship is:

$$\mathbf{s}(t + dt) = \begin{cases} x(t + dt) = x(t) + v_x(t)dt, \\ y(t + dt) = y(t) + v_y(t)dt, \\ \theta(t + dt) = \theta(t) + \omega(t)dt, \\ v_x(t + dt) = v_x(t) + a(t) \cos \theta(t)dt, \\ v_y(t + dt) = v_y(t) + a(t) \sin \theta(t)dt, \end{cases} \quad (3)$$

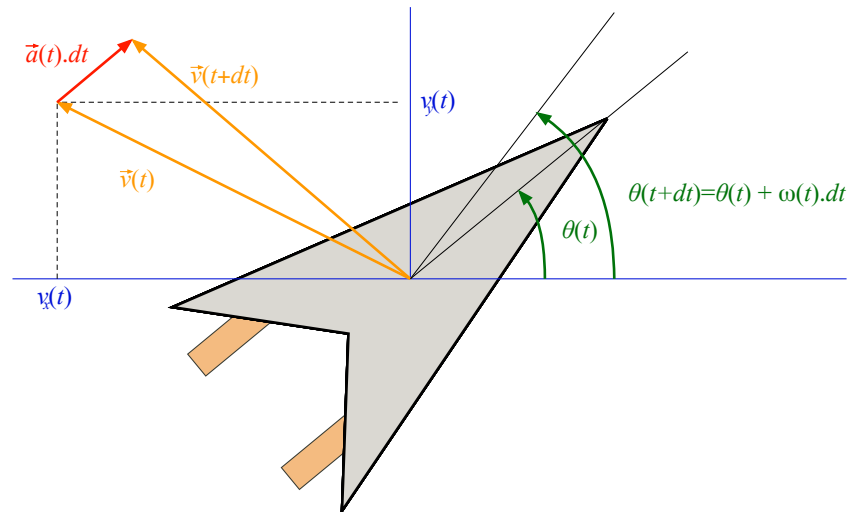


Figure 6: The controls applicable by the user to the spaceship.

where the *control parameters* $a(t)$ and $\omega(t)$ are selected by the user.

At this point, purists may argue³ that on a real spaceship,

1. We couldn't control directly acceleration but thrust;
2. We could even less control the angular velocity $\omega(t)$, only apply thrust one way or another to accelerate or decelerate spin.

Both objections are valid. I will simply reply that

- Thrust equals mass times acceleration. So, as long as the mass of the spaceship remains more or less constant, we can consider that the player is directly controlling acceleration (as in, we had better hope that the mass of the spaceship doesn't decrease by more than a tiny fraction of a percent within the few seconds of gameplay).
- For any simulation taking place within an application, there is always a level of sophistication of the computational model (be it physics, economy, ecology, psychology, etc.) that is “good enough” for this kind of application. So, whether the spaceship is being simulated for a silly 2D asteroid game, a “Space Simulator” game, or mission planning by a space agency, a different level of complexity of the model will be more appropriate, but the general idea of the simulation remains the same: The state equation models the behavior of the system.
- I will offer extra credit for a refined spin control model.

³And they would be right about it, which wouldn't make them any less annoying.

3.3 Implementation of the controls

The user should be able to control the spaceship using the following key action:

- '←' and 'a' set constant counterclockwise angular velocity w to the spaceship;
- '→' and 'd' set constant clockwise angular velocity w to the spaceship;
- '↑' and 'w' apply thrust (constant value a) to the spaceship;
- mouse clicks and the space bars both can be used to shoot projectiles (more on that later).

Note that there is no “slow down” or “decelerate” control: The only way for the spaceship to go slower in the same direction is to rotate by 180 degree and fire its boosters against its current direction of travel.

3.4 Collision with asteroids

3.4.1 Game aspect (expected behavior)

Each time the spaceship collides with an asteroid, it should take some structural damage. At some level of accumulated structural damage, the spaceship should lose half, then all of its thrust power (at this point, it could only spin and shoot). Eventually, if the ship takes too many hits, it should be destroyed.

For this assignment, we are going to consider that the spaceship collided with an asteroid if we detect that bounding boxes intersect at the finest part level of the two object. In other words, we won't go down to fully implementing an intersection test for convex polygons.

It's up to you to fix appropriate values for the damage resulting from a collision, and the different levels at which the spaceship loses functionality.

3.4.2 Collision detection

As discussed below (Subsection 4.3), our asteroids are simple shapes, with a single bounding box. Our spaceship, however, is made up of parts, each of which should have its own bounding box. And then there should be a global bounding box for the object. Whether you choose “absolute” or “relative” bounding boxes is up to you. Absolute bounding boxes require updating every time the spaceship (resp. asteroid) rotates, but then all bounding boxes are straight rectangles in the original XY reference frame, which makes collision detection easy. Relative bounding boxes are fixed, so no update required if the object rotates, but they all have their own reference frame (orientation) and change of coordinates transformations must be applied in order to test for collision.

Whichever type of box you choose, I want you to implement a *hierarchical* collision detection: First check the asteroid's box against the spaceship's *global* box. If the two boxes collide, then check the asteroid's box against that of the spaceship parts.

3.5 Missiles

Your spaceship shoots missiles/projectiles/bullets with the following characteristics:

- The projectiles are small enough that they can be treated as points for the purpose of collision detection.
- The projectiles are shot in the current heading direction of the spaceship at the time they are shot.
- The projectile's initial velocity is relative to that of the spaceship (this means that you need to add up the two velocity vectors to get the absolute velocity vector of the projectile in world coordinates).
- The spaceship can only shoot a given number of projectiles per second (i.e. it cannot shoot a cloud of bullets that destroy everything on their way).
- The projectiles “wrap around” the window in the same fashion as the asteroids and the spaceship, but they cannot travel longer than a given amount of time. After that, they simply disappear.

Again, it's up to you to set suitable values for the speed of the projectiles, their “time limit,” etc.

4 Part III: The Environment

4.1 World model

For this application, you should use a “spherical” world model, as we saw in class (and handouts). This means that objects reaching one boundary of the window should start to reappear on the opposite side of the window. Feel free to reuse any of the code that I have posted. You are all joint coauthors of this code since we did it in class together.

4.2 Alternative world

If you object to the notion of shooting bullets at stuff, even chunks of rock, then feel free to replace the “asteroids” setup of this assignment by something more peaceful, like throwing candy. As long as the general mechanics of the game and constraints on behavior of the objects are preserved, I am fine with any alternative narrative to slap on them.

4.3 The asteroids

For this assignment, I am mostly interested in the control and collision detection for the asteroid (and the Frogger before that), so we are going to keep the asteroids simple: An ellipse or a rectangular box will do, as long as they have a bit of spin applied to them.

5 Extra credit

5.1 Extra credit 1 (up to 4 points)

Create a really good-looking spaceship. Don't wait too much effort on this, keeping in mind one of the mottos of 3D artists: Don't model what you can texture. In other words, there is no point trying to draw a complex shape when a nice image looks much better (as long as your bounding boxes match the nice image). We haven't talked about texture mapping in class yet, but there is no reason not to take this good advice in advance, particularly when it dispenses us from doing more work.

5.2 Extra credit 2: Score and health (4 pts)

Your program should display the score, number of remaining lives, and current level of structural integrity in a corner of the screen.

5.3 Extra credit 3 (3 points)

Display graphically (rather than as numbers) the remaining number of lives and structural integrity level.

5.4 Extra credit 4 (4 points)

Display some exhaust flames when you apply thrust to your spaceship.

5.5 Extra credit 8 (8 points)

Add another control for the display to toggle between “geocentric” and “egocentric” views.

- ‘g’ switches to geocentric view. This is the view we have been using so far: The world is fixed and the spaceship (and obstacles) move relative to it.
- ‘e’ switches to egocentric view. In this view the spaceship stays in the center of the window, oriented horizontally, and the obstacles (and implicitly the world) move relative to the spaceship. So, in this view, applying an angular velocity to the spaceship results in the obstacles spinning around the—seemingly static—spaceship.

6 What to Hand in

Commented source files (one source file for each class, one separate header file for each class in C++), including javadoc-style comments for all classes, variables, and functions in your code. A report that explains the structure of your classes and whatever design decision you may have had to make to complete the assignment.

7 Grading

If you submit code that could not possibly compile (contains obvious syntax errors) your assignment will not be graded and you will get a grade of 0. For all others:

- the code does what it is supposed to do: 40%
- good class design: 15%
- comments: 15%
- general code quality: 15%
- report: 15%